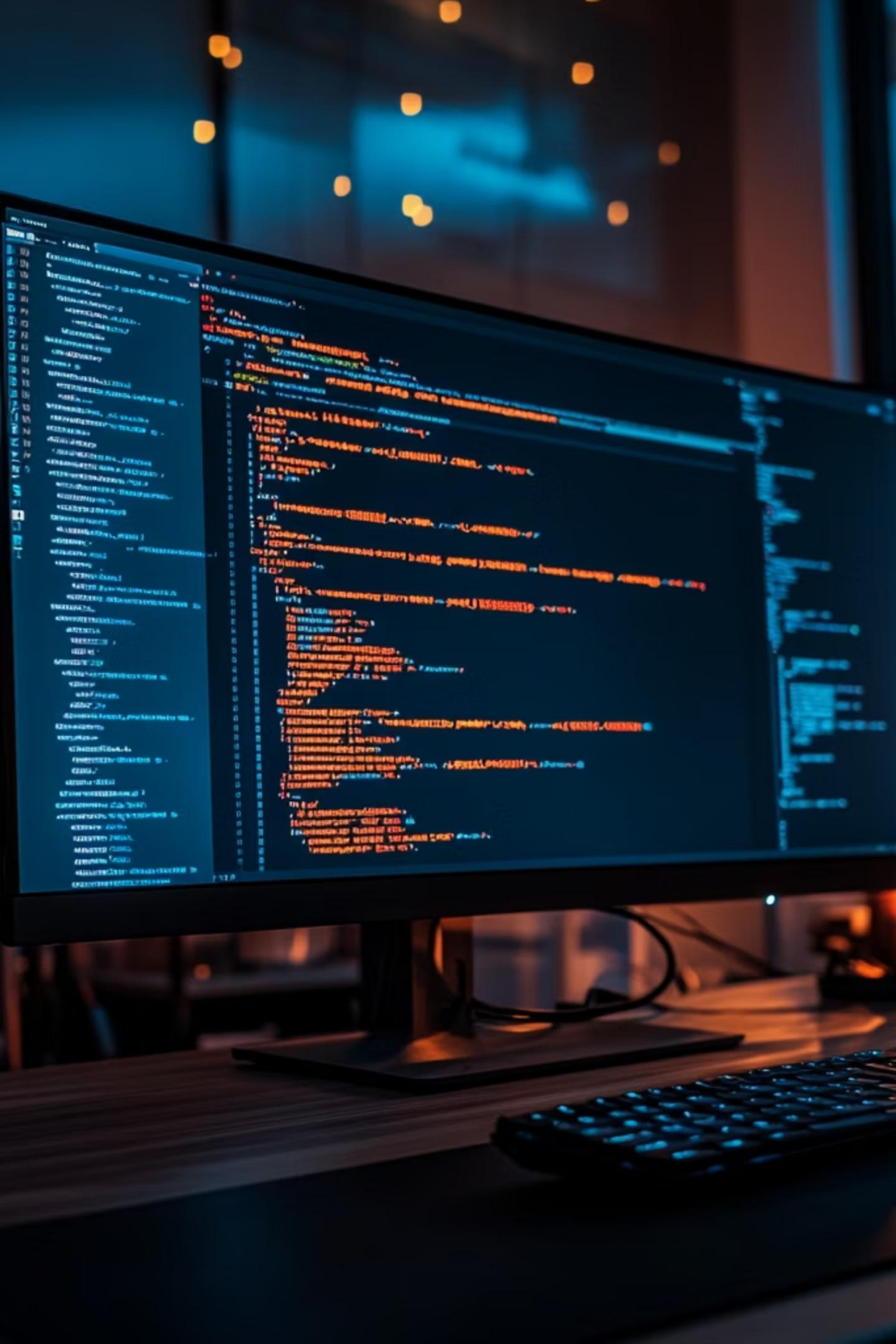


Codificação Para Front-end

15

2026

SENAI - CRTI



Variáveis, Animações e Organização de Código

Dando continuidade ao nosso e-commerce, chegou a hora de elevar a qualidade do projeto com padronização visual, animações suaves e código limpo e escalável. Vamos transformar um site funcional em um produto profissional.

Situação de Aprendizagem

O E-commerce Funciona — Mas Pode Ser Muito Melhor

Nas aulas anteriores, construímos um e-commerce **funcional e responsivo**. Isso é uma conquista enorme! Mas ao olhar o código com mais atenção, percebemos alguns problemas comuns que aparecem em projetos reais:

- Cores repetidas em dezenas de lugares
Mudar o tom principal exige editar o arquivo inteiro.
- Ausência de feedback visual
Botões e cards não reagem às ações do usuário.
- Código sem estrutura lógica
Difícil encontrar, manter e escalar com o tempo.

Missão da Aula

Aplicar **variáveis CSS, animações e boas práticas de organização** para transformar o e-commerce em um projeto de nível profissional, pronto para evoluir com JavaScript.

CAPACIDADES

O Que Você Vai Desenvolver Nesta Aula



Variáveis CSS

Usar **custom properties** para centralizar cores, fontes e espaçamentos, tornando o código fácil de manter e atualizar.



Organização de Código

Estruturar o CSS em blocos lógicos e comentados, seguindo **padrões profissionais** de desenvolvimento.



Animações e Transições

Aplicar **transition** e **@keyframes** para criar microinterações que melhoram a experiência do usuário.



Introdução ao SASS/SCSS

Conhecer de forma **conceitual** os pré-processadores CSS e entender como eles potencializam o desenvolvimento.

O Problema: Código Repetitivo e Difícil de Manter

✗ Antes: Código Problemático

```
/* Cor primária repetida em 30 lugares */  
.header { background: #0570b5; }  
.btn-primary { background: #0570b5; }  
.link:hover { color: #0570b5; }  
.badge { border: 2px solid #0570b5; }  
  
/* Mudar a cor = editar tudo isso! */
```

Imagine que o cliente pede para mudar a cor principal. Você teria que encontrar e substituir manualmente cada ocorrência — e correr o risco de errar.

✓ Depois: Com Variáveis CSS

```
/* Definido uma única vez */  
:root {  
  --cor-primaria: #0570b5;  
}  
  
/* Usado em qualquer lugar */  
.header { background: var(--cor-primaria); }  
.btn-primary { background: var(--cor-primaria); }  
.link:hover { color: var(--cor-primaria); }  
  
/* Mudar a cor = editar 1 linha! */
```

Com variáveis, uma única alteração no `:root` reflete em todo o projeto instantaneamente.

CSS Variables: O Que São e Como Funcionam

As **Custom Properties** (variáveis CSS) são valores reutilizáveis que você define uma vez e usa em todo o projeto. São declaradas com dois hifens (--) e acessadas com a função `var()`.

Declaração

```
:root {  
  --cor: #0570b5;  
}
```

Declarada no `:root` para escopo global.

Uso

```
.btn {  
  background:  
    var(--cor);  
}
```

Referenciada com a função `var()`.

Fallback

```
.btn {  
  color:  
    var(--cor, blue);  
}
```

Valor padrão caso a variável não exista.

Dinamismo

```
/* Funciona com JS! */  
el.style.setProperty(  
  '--cor', '#red'  
);
```

Pode ser alterada via JavaScript em tempo real.

  Diferente de variáveis SASS, as CSS Variables **existem no navegador** e podem ser manipuladas com JavaScript, abertura poderosa para temas dinâmicos.

Definindo Variáveis no :root do E-commerce

Veja como estruturar o `:root` do nosso e-commerce de forma profissional, separando as variáveis por categoria para facilitar a leitura e manutenção:

```
:root {  
  /* === CORES === */  
  --cor-primaria: #0570b5;  
  --cor-secundaria: #02294d;  
  --cor-destaque: #51c8e6;  
  --cor-fundo: #f8f9fa;  
  --cor-texto: #333333;  
  --cor-texto-claro: #666666;  
  
  /* === TIPOGRAFIA === */  
  --fonte-principal: 'Inter', sans-serif;  
  --tamanho-base: 1rem;  
  --tamanho-titulo: 2rem;  
  --peso-normal: 400;  
  --peso-negrito: 700;  
  
  /* === ESPAÇAMENTOS === */  
  --espaco-sm: 0.5rem;  
  --espaco-md: 1rem;  
  --espaco-lg: 2rem;  
  --espaco-xl: 4rem;  
  
  /* === BORDAS === */  
  --borda-raio: 8px;  
  --borda-raio-lg: 16px;  
  --sombra-padrao: 0 2px 8px rgba(0,0,0,0.1);  
}
```

Desafio Rápido

Após definir as variáveis, tente alterar apenas o valor de `--cor-primaria` para outra cor. Observe como **todo o e-commerce muda automaticamente!**

✔️ Isso é o poder da centralização: um ponto de verdade para todo o visual do projeto.

Boas Práticas de Nomenclatura

- Use nomes semânticos: `--cor-primaria`, não `--azul`
- Use hifens para separar palavras
- Agrupe por categoria com comentários
- Prefixe variáveis de componentes: `--btn-bg`

Transições CSS: Movimento com Propósito

A propriedade `transition` cria uma **animação suave** entre dois estados de um elemento — por exemplo, quando o usuário passa o mouse sobre um botão. São essenciais para uma boa experiência visual.

Sintaxe da Transition

```
/* Sintaxe completa */
transition: propriedade duração
           timing-function delay;

/* Exemplo no botão do e-commerce */
.btn-comprar {
  background: var(--cor-primaria);
  transform: scale(1);
  box-shadow: var(--sombra-padrao);
  transition: background 0.3s ease,
             transform 0.2s ease,
             box-shadow 0.3s ease;
}

.btn-comprar:hover {
  background: var(--cor-secundaria);
  transform: scale(1.03);
  box-shadow: 0 6px 16px rgba(0,0,0,0.2);
}
```

Funções de Timing

`ease`

Inicia rápido, desacelera no fim. Padrão e natural.

`linear`

Velocidade constante do início ao fim.

`ease-in-out`

Começa e termina devagar. Muito elegante.

`cubic-bezier`

Controle total com curva personalizada.

🕒 Duração ideal: entre **150ms e 400ms**. Muito rápido parece brusco; muito lento parece travado.

Animações com @keyframes: Controle Total do Movimento

Enquanto `transition` anima entre dois estados, o `@keyframes` permite criar animações complexas com **múltiplos estágios**, loops e controle preciso de cada frame da animação.

```
/* 1. Definindo a animação */
@keyframes fadeIn {
  from {
    opacity: 0;
    transform: translateY(20px);
  }
  to {
    opacity: 1;
    transform: translateY(0);
  }
}

/* 2. Aplicando no elemento */
.produto-card {
  animation: fadeIn 0.5s ease forwards;
}

/* Animação de pulso no badge "Novo" */
@keyframes pulso {
  0%, 100% { transform: scale(1); }
  50%      { transform: scale(1.08); }
}

.badge-novo {
  animation: pulso 1.5s ease-in-out
infinite;
}

/* Skeleton loading (UX moderno) */
@keyframes shimmer {
  0%   { background-position: -200px 0; }
  100% { background-position: 200px 0; }
}
```

Propriedades de Animation

`animation-name`

Nome do @keyframes a usar.

`animation-duration`

Quanto tempo dura um ciclo.

`animation-timing-function`

Curva de velocidade.

`animation-delay`

Tempo antes de iniciar.

`animation-iteration-count`

Quantas vezes repete. Use `infinite` para loop.

`animation-fill-mode`

`forwards` mantém o estado final da animação.

UX em Ação: Microinterações que Encantam

Microinterações são pequenas animações que dão feedback visual ao usuário, tornando a navegação mais intuitiva e agradável. São o detalhe que separa um site amador de um produto profissional.

Botões Responsivos

Escala sutil + mudança de cor ao hover. O usuário percebe que o elemento é clicável antes mesmo de clicar, reduzindo confusão e aumentando conversão.

Cards de Produto

Elevação com `box-shadow` e leve `translateY(-4px)` ao hover. Cria sensação de profundidade e destaca o produto de interesse do usuário.

Campos de Formulário

Borda colorida e brilho suave ao focar em campos de busca ou login. Guia o olhar e confirma ao usuário que o campo está ativo e pronto para digitação.

Fade-in na Entrada

Produtos aparecem suavemente ao carregar a página com `@keyframes fadeIn`. Elimina a sensação de "piscada" e torna a experiência mais fluida e polida.

Organização de CSS: Código que Escala

Um CSS bem organizado é tão importante quanto um CSS correto. Veja a estrutura recomendada para nosso e-commerce:

```
/* 1. VARIÁVEIS GLOBAIS */
:root { ... }

/* 2. RESET E BASE */
*, *::before, *::after {
  box-sizing: border-box;
}
body { ... }

/* 3. TIPOGRAFIA */
h1, h2, h3, p { ... }

/* 4. LAYOUT PRINCIPAL */
.container { ... }
.header { ... }
.nav { ... }

/* 5. COMPONENTES */
/* -- Botões -- */
.btn { ... }
.btn-primario { ... }

/* -- Cards -- */
.produto-card { ... }

/* -- Formulários -- */
.form-busca { ... }

/* 6. SEÇÕES DA PÁGINA */
.secao-hero { ... }
.secao-produtos { ... }
.secao-footer { ... }
```

```
/* 7. ANIMAÇÕES */
@keyframes fadeIn { ... }
@keyframes pulso { ... }

/* 8. RESPONSIVIDADE */
@media (max-width: 768px) { ... }
```

Princípios de Organização

- 1 Do Geral para o Específico
Variáveis → Reset → Layout → Componentes → Páginas.
Hierarquia clara facilita a manutenção.
- 2 Comentários como Separadores
Use blocos de comentários para dividir seções. Facilita a navegação no arquivo.
- 3 Uma Responsabilidade por Classe
Evite classes que fazem tudo. Prefira classes pequenas e combináveis.
- 4 Nomenclatura Consistente
Use um padrão (BEM, OOCSS ou o seu próprio) e mantenha-o em todo o projeto.

Introdução ao SASS/SCSS: O Próximo Nível

O **SASS (Syntactically Awesome Style Sheets)** é um pré-processador CSS que adiciona recursos poderosos à linguagem — variáveis (antes das nativas), aninhamento, mixins e funções. O arquivo `.scss` é compilado para CSS puro antes de ir ao navegador.

Variáveis SASS

```
$cor-primaria: #0570b5;
.btn {
  background: $cor-primaria;
}
```

Sintaxe com `$`, compiladas no build.

Aninhamento

```
.card {
  padding: 1rem;
  &:hover {
    box-shadow: ...;
  }
  .titulo {
    font-size: 1.2rem;
  }
}
```

Hierarquia visual no código.

Mixins

```
@mixin flex-center {
  display: flex;
  align-items: center;
  justify-content: center;
}
.container {
  @include flex-center;
}
```

Blocos de código reutilizáveis.

Partials

```
/* _variaveis.scss */
/* _botoes.scss */
/* _cards.scss */

/* main.scss */
@use 'variaveis';
@use 'botoes';
```

Divide o CSS em múltiplos arquivos.

Mãos na Massa: Refatorando o E-commerce

Agora é a sua vez! Aplique tudo que aprendemos diretamente no projeto. Esta atividade deve ser realizada no arquivo CSS do seu e-commerce.



Passo 1: Criar as Variáveis

Defina no `:root` variáveis para: cores principais (primária, secundária, destaque, fundo, texto), fontes e tamanhos, espaçamentos padrão e raios de borda.



Passo 3: Adicionar Transições

Aplique `transition` nos botões de compra, nos cards de produto ao hover, e nos links do menu de navegação.



Passo 2: Substituir Valores Fixos

Percorra todo o CSS e substitua cores e valores repetidos pelos `var()` correspondentes. Teste se o visual permanece idêntico ao original.



Passo 4: Criar Animação Fade-in

Crie um `@keyframes fadeIn` e aplique nos cards de produto para que apareçam suavemente quando a página carrega.



Desafio Extra: Formate as demais seções da sua página, de forma consistente, mantendo a identidade visual.

Resumo: O Que Aprendemos Hoje



CSS Variables

Custom properties com `--nome` no `:root`, acessadas com `var()`. Um ponto de verdade para todo o visual.



Transitions

Animação entre dois estados com controle de duração e timing. Essenciais para hover em botões, cards e links.



@keyframes

Animações complexas com múltiplos estágios, delays e loops. Usadas para fade-in, pulso e skeleton loading.



Organização CSS

Estrutura em seções comentadas: variáveis → reset → layout → componentes → animações → responsividade.



SASS/SCSS

Pré-processador que potencializa o CSS com aninhamento, mixins e partials. Base para projetos de grande escala.



Microinterações UX

Pequenas animações que guiam o usuário, confirmam ações e criam uma experiência fluida e profissional.

